

APPLICATIONS: REMOTE LOGIN (TELNET)

Reti di Calcolatori (Modulo 2)

Introduction

- ▶ High-level services provide more communication functionalities
- ▶ They allow to interact with remote machines and users
- ▶ High-level protocols are implemented with application programs

Remote Interaction

- ▶ Deviced when computers used (ASCII) terminals
- ▶ Terminal abstraction extended to remote access over a network
- ▶ TCP makes possible interactive use of remote machines
 - ▶ E.g., remote text editor

Client-Server Interaction

▶ Client

- ▶ Invoked by user
- ▶ Forms a TCP connection to remote server
- ▶ Passes keystrokes from user's keyboard to server and displays output from server on user's screen

▶ Server

- ▶ Accepts connection over the network
- ▶ Passes incoming characters to OS as if they were typed on a local keyboard
- ▶ Sends output over connection to client

Client-Server Interaction

- ▶ Not always suitable/possible
 - ▶ A server for each specialized service overload the server machine
- ▶ A more general solution provides remote login facilities
 - ▶ Establish a login session and execute commands on the remote machine
 - ▶ System designers do not need to provide specialized servers

Remote login

- ▶ Providing remote login is not a simple task
 - ▶ For machine not ready for networking, it requires change to the OS
 - ▶ Compatibility problem for client software, CTRL+C may be interpreted either as «abort the currently executing command process» or as a command to be passed to the remote site
- ▶ Remote login available for most OS
 - ▶ Client software overrides local interpretation of keys
 - ▶ Exception: a way for a user to escape to the local environment and control the client (e.g., abort the client)

Remote login

- ▶ Some remote login protocols
 - ▶ Define *trusted host* for which no authentication is required
 - ▶ Encrypt each communication → Non per Teher

TELNET: Introduction

- ▶ Standard protocol for remote terminal access (TCP/IP protocol suite)
- ▶ Allow users to log on PC across the Internet
 - ▶ TCP connection
 - ▶ Pass keystrokes from user to server
 - ▶ Display output on the user screen
 - ▶ *Transparent*: the user has the impression of acting on the local machine

TELNET: Advantages

- ▶ Simple
- ▶ Widely available
- ▶ Works both with IP addresses and domain name
 - ▶ Robust against DNS failures

TELNET: Services

▶ Three basic services

1. Defines *Network Virtual Terminal* (NVT) that provides standard interface to remote systems
2. Mechanism that allows client and server to negotiate options (e.g., standard 7-bit ASCII character set)
3. Symmetric treatment that allows either end of the connection to be a program instead of a physical keyboard and display
 - ▶ Arbitrary program to become a client

Illustration Of TELNET

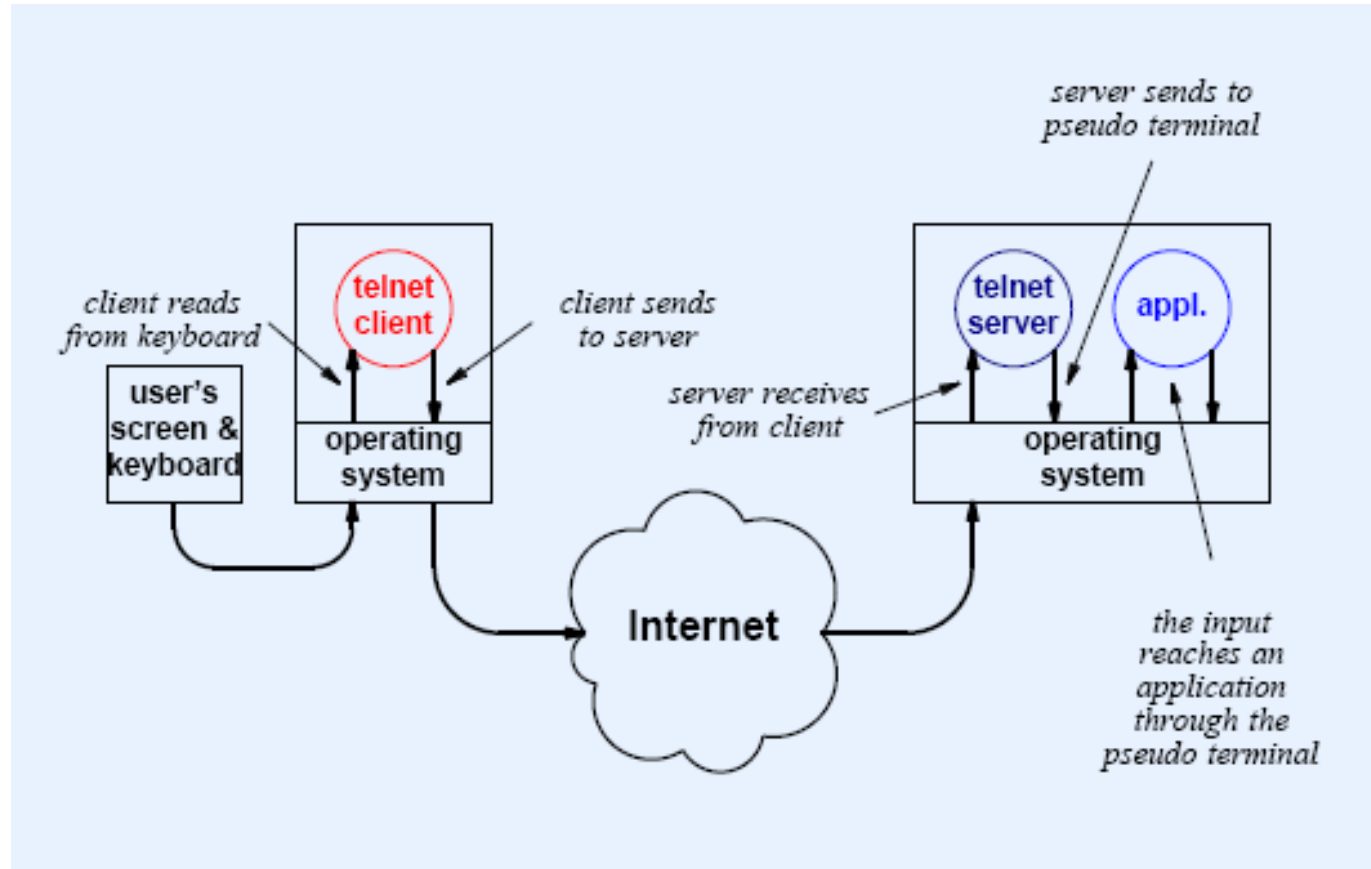


Illustration Of TELNET

- ▶ A user invokes TELNET and an application program becomes the client
- ▶ Client and server establish a TCP connection
- ▶ Client sends data that are received by the server via the TCP connection and forwarded to the local OS
- ▶ Server sends back data displayed by the client

Illustration Of TELNET

- ▶ Server must support and handle multiple parallel connections
- ▶ A master server process waits for new connections
- ▶ A slave process is created for each new connection

Illustration Of TELNET

- ▶ *Pseudo terminal*: operating system entry point allowing TELNET server to transfer data to the OS as if they came from a keyboard
- ▶ Pseudo terminal abstraction allows to build TELNET with application programs
- ▶ Each slave connects a TCP stream from one client to the pseudo terminal down to the OS

Illustration Of TELNET

- ▶ TELNET as an application program
 - ▶ Advantage
 - ▶ Modification and control of the server easier than if it was coded in the OS
 - ▶ Disadvantage
 - ▶ Inefficiency: each keystroke travels around OSs, applications, Internet
 - ▶ Several process context switch required

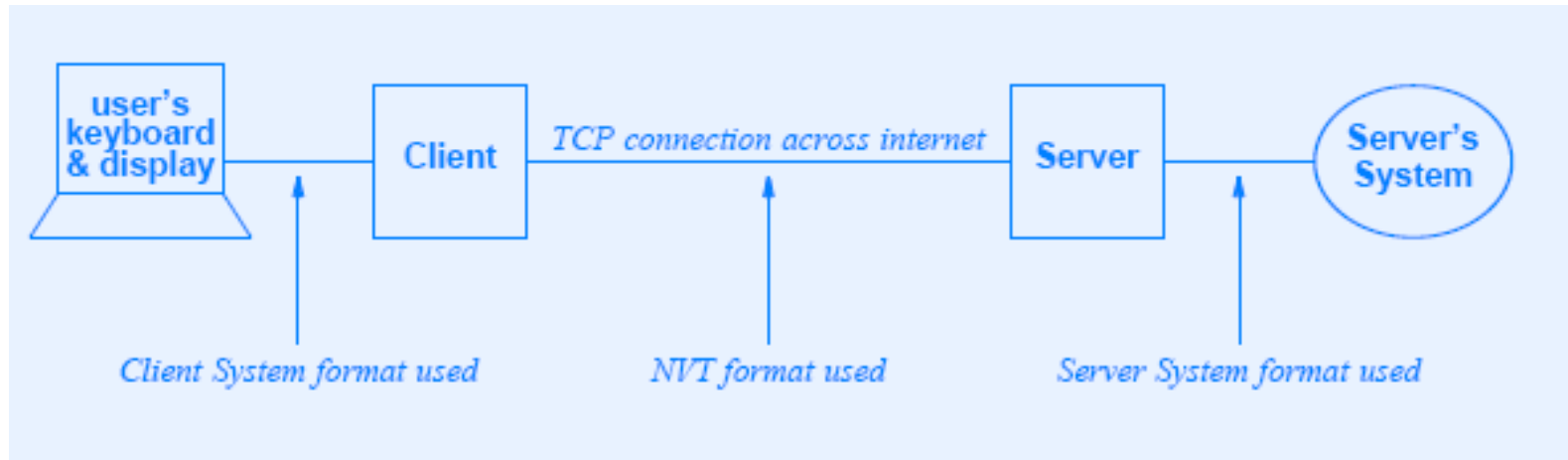
Accommodating Heterogeneity

- ▶ PROBLEM: Interoperability, heterogeneous computers and operating systems
- ▶ Different systems use different encoding
 - ▶ CR, LF, CR+LF
 - ▶ CTRL+C, ESC
- ▶ TELNET must accomodate the details of many computers and OSs

Accommodating Heterogeneity

- ▶ TELNET defines how data and command are sent across the Internet
- ▶ *Network Virtual Terminal (NVT) describes system independent encoding*
- ▶ TELNET client and server map NVT into local computer's representation

Illustration Of How NVT Accommodates Heterogeneity



Definition of NVT

- ▶ Straightforward
- ▶ Communication involves 8-bit bytes
- ▶ At startup
 - ▶ Standard 7-bit USASCII representation
 - ▶ Bytes with 8th-bit set to 1 are for commands
 - ▶ USASCII has 95 characters and 33 control codes
 - ▶ 33 control codes used by NVT: control character interpretation
 - ▶ Standard line termination of NVT is CR+LF

Definition of TELNET NVT

- ▶ The telnet NVT interpretation of USASCII Control characters

ASCII Control Code	Decimal Value	Assigned Meaning
NUL	0	No operation (has no effect on output)
BEL	7	Sound audible/visible signal (no motion)
BS	8	Move left one character position
HT	9	Move right to the next horizontal tab stop
LF	10	Move down (vertically) to the next line
VT	11	Move down to the next vertical tab stop
FF	12	Move to the top of the next page
CR	13	Move to the left margin on the current line
<i>other control</i>	—	No operation (has no effect on output)

Control functions

- ▶ The systems reserve characters or character sequences for control functions
 - ▶ E.g., CTRL+C to terminate a running application
- ▶ TELNET NVT defines how they are passed from the client to the server
 - ▶ User's keyboard has virtual keys for functions used to control processing
 - ▶ E.g., NVT defines a conceptual interrupt key that requests program termination

TELNET NVT Control Functions

Signal	Meaning
IP	Interrupt Process (terminate running program)
AO	Abort Output (discard any buffered output)
AYT	Are You There (test if server is responding)
EC	Erase Character (delete the previous character)
EL	Erase Line (delete the entire current line)
SYNCH	Synchronize (clear data path until TCP urgent data point, but do interpret commands)
BRK	Break (break key or attention signal)

TELNET NVT Control Functions

- ▶ Most keyboards do not provide extra keys for commands
- ▶ Individual operating systems or command interpreters have a variety of ways to generate them
 - ▶ OSs bind an ASCII character to a control function
- ▶ Commands are separated from ASCII characters in NVT
 - ▶ More flexibility
 - ▶ Can transfer all possible data and control functions
 - ▶ Signals specified unambiguously

TELNET NVT Control Functions

- ▶ *Escape sequence* to pass control functions
- ▶ An escape sequence uses a reserved octet to indicate a control code
 - ▶ *Interpret as command* (IAC) octet
- ▶ Example: program termination
 - ▶ Two octet sequence IAC IP (255-244)

TELNET Commands

Command	Decimal Encoding	Meaning
IAC	255	Interpret next octet as command (when the IAC octet appears as data, the sender doubles it and sends the 2-octet sequence IAC-IAC)
DON'T	254	Denial of request to perform specified option
DO	253	Approval to allow specified option
WON'T	252	Refusal to perform specified option
WILL	251	Agreement to perform specified option
SB	250	Start of option subnegotiation
GA	249	The "go ahead" signal
EL	248	The "erase line" signal
EC	247	The "erase character" signal
AYT	246	The "are you there" signal
AO	245	The "abort output" signal
IP	244	The "interrupt process" signal
BRK	243	The "break" signal
DMARK	242	The data stream portion of a SYNCH (always accompanied by TCP Urgent notification)
NOP	241	No operation
SE	240	End of option subnegotiation
EOR	239	End of record

TELNET Commands

- ▶ TELNET commands represent the way to send NVT control functions
 - ▶ The signals generated by conceptual keys on an NVT keyboard refer to corresponding commands
- ▶ If they refer to control functions, send the standard control functions
- ▶ Additional commands are proposed, for instance, to negotiate options between client and server

TELNET Control Sequences And TCP

- ▶ Problem: application on the remote server in an infinite loop
 - ▶ User sends an *interrupt process* control function and wants the server to stop the application
 - ▶ Operating system buffer will fill
 - ▶ Server does not read data on TCP connection (window of zero length)
 - ▶ The control function never reaches the server

TELNET Control Sequences And TCP

- ▶ Problem: application on the remote server in an infinite loop
 - ▶ User sends an *interrupt process* control function and wants the server to stop the application
 - ▶ Operating system buffer will fill
 - ▶ Server does not read data on TCP connection (window of zero length)
 - ▶ The control function never reaches the server
- ▶ Solution: use TCP's *urgent data* to send control sequences
 - ▶ Out of band signaling

TELNET Control Sequences And TCP

- ▶ A SYNCH control function is sent
 - ▶ A Synch signal consists of a TCP Urgent notification, coupled with the TELNET command DATA MARK
 - ▶ IAC "user signal" (command) followed by "IAC DMARK" (come back to normal operation)
 - ▶ E.g., IAC IP IAC DMARK: an interrupt process signal followed by a data mark.
- ▶ TELNET appends a reserved octet *data mark*
 - ▶ The TELNET command DATA MARK (DM) is the synchronizing mark in the data stream which indicates that any special signal has already occurred and the recipient can return to normal processing of the data stream

TELNET Control Sequences And TCP

- ▶ TCP signals the server sending a segment with URGENT DATA bit set
 - ▶ Urgent notification, which is not subject to the flow control pertaining to the TELNET connection, is used to invoke special handling of the data stream by the process which receives it
 - ▶ This segment reaches the server immediately (socket option SO_OOBLINE)
 - ▶ The data stream is immediately scanned for "interesting" signals, discarding intervening data
 - ▶ The server reads and discards all data until the data mark is found
- ▶ "Interesting" signals are
 - ▶ The TELNET standard representations of IP, AO, and AYT (but not EC or EL)
 - ▶ The local analogs of these standard representations (if any)
 - ▶ All other TELNET commands

TELNET Control Sequences And TCP

- ▶ *TELNET cannot rely on the conventional data stream alone to carry control sequences between client and server because a misbehaving application that needs to be controlled might inadvertently block the data stream*

TELNET Options

- ▶ Options are negotiable
- ▶ Client and server can reconfigure their connection
- ▶ Range of options is wide
 - ▶ Pass from 7-bit to 8-bit communication
 - ▶ IAC command must be doubled
 - ▶ Half-duplex or full-duplex communication
 - ▶ Determine user's terminal type

TELNET Options

Name	Code	RFC	Meaning
Transmit Binary	0	856	Change transmission to 8-bit binary
Echo	1	857	Allow one side to echo data it receives
Suppress-GA	3	858	Suppress (no longer send) Go-ahead signal after data
Status	5	859	Request for status of a TELNET option from remote site
Timing-Mark	6	860	Request timing mark be inserted in return stream to synchronize two ends of a connection
Terminal-Type	24	884	Exchange information about the make and model of a terminal being used (allows programs to tailor output like cursor positioning sequences for the user's terminal)
End-of-Record	25	885	Terminate data sent with EOR code
Linemode	34	1116	Use local editing and send complete lines instead of individual characters



TELNET Option Negotiation

- ▶ Both ends can start a request or ask for an option (symmetric negotiation)
 - ▶ The receiving end can accept or refuse
- ▶ Negotiation
 - ▶ WILL X request: “will you agree to let me use option X?”
 - ▶ DO or DON’T X response
- ▶ Negotiation symmetry
 - ▶ DO X request
 - ▶ WILL or WON’T X response: “I will/won’t start using option X”

TELNET Option Negotiation

- ▶ The *TERMINAL-TYPE* option requires some way for the client to send the server the type of the terminal
- ▶ Suppose the server supports this option and would like the client to use it
 - ▶ IAC DO TERMINAL-TYPE
- ▶ If the client agrees, it will respond
 - ▶ IAC WILL TERMINAL-TYPE

TELNET Option Negotiation

- ▶ *TELNET uses a symmetric option negotiation mechanism to allow clients and servers to reconfigure the parameters controlling their interaction*
- ▶ *Because all TELNET software understands a basic NVT protocol, clients and servers can interoperate even if one understands options another does not*
 - ▶ *A communication can always revert to the default configurations*

Remote Login (rlogin)

- ▶ Invented for BSD Unix
 - ▶ Includes facilities for Unix
 - ▶ Support trusted hosts
- ▶ Allows managers to configure a set of computers so that login names and file access protections are shared
- ▶ Permits other forms of authentication (login equivalences)
 - ▶ User A has login X and login Y on two machines
 - ▶ Can login on a remote machine without typing a password

Remote Login (rlogin)

- ▶ Remote shell (rsh) is a variant of rlogin
 - ▶ Invokes a command interpreter passing command line arguments (no login)
 - ▶ It can be used in programs as well as from the keyboard
 - ▶ Also part of BSD Unix
 - ▶ Allows remote execution of a single command
 - ▶ *rsh machine command*

Remote Login (rlogin)

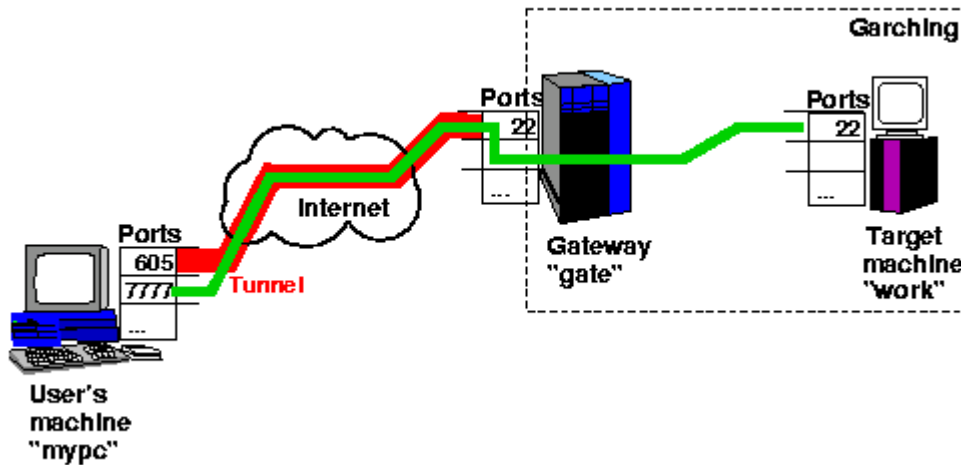
- ▶ Understand both remote and local environments
 - ▶ *rlogin* understands the notions of *stin*, *stout*, *sterr* and can connect them to the remote host
 - ▶ *rlogin* understands control functions
 - ▶ *rlogin* exports some user's environment information to the server

Secure Remote Login (ssh)

- ▶ (Better) Alternative to TELNET/rlogin
- ▶ Transport layer protocol with service authentication
- ▶ User authentication protocol
- ▶ Connection protocol
 - ▶ Multiplexes multiple transfers
 - ▶ Uses encryption for privacy

Port Forwarding

- ▶ A fundamental aspect of SSH
- ▶ Similar to NAT
- ▶ Permits incoming TCP connection to be forwarded across secure tunnel



Remote Desktop

- ▶ Intended for systems that have a GUI interface
- ▶ Allows a remote user to see screen of remote system and use mouse as well as keyboard
- ▶ Examples include
 - ▶ Virtual Network Computing (VNC)
 - ▶ Remote Desktop Protocol (RDP)

Summary

- ▶ Remote interaction allows client software to connect local keyboard and screen to remote system
- ▶ Standard protocol is TELNET
- ▶ Alternatives include *rlogin*, *rsh*, and *ssh*
- ▶ Remote desktop extends remote access to handle GUI interface





Summary



Summary

Telnet

- ▶ Allows users to connect to a remote host
 - ▶ Based on TCP
- ▶ Transmits data in NVT (Network Virtual Terminal) format
 - ▶ Uses extended ASCII
- ▶ Requires authentication via username and password
- ▶ Requires a telnet server (telnetd) to be running on the remote system

Telnet

- ▶ Send the sequences of keystrokes typed on the client to the remote system and displays the output generated by the remote system on the user's screen
- ▶ Provides two basic services
 - ▶ A virtual network terminal, the standard interface to the remote system
 - ▶ A mechanism for managing communication options, treating both ends of a connection symmetrically

Telnet

- ▶ It's not secure!
- ▶ The TELNET process on the client connects to the TELNET on the server
 - ▶ `telnet crema.unimi.it`
- ▶ The server listens for incoming connections on the well-known port 23
 - ▶ Subsequently, the server forks a new slave process to handle the connection.

Why Telnet?

- ▶ The main task in Telnet is to manage system heterogeneity
 - ▶ Including control characters, etc
 - ▶ Executed through the Network Virtual Terminal (NVT) specification for ASCII
- ▶ It can cause a lot of traffic (tinygram)
 - ▶ Uses the Nagle algorithm (RFC 896, 1984) to combine and send groups of data based on the ACK process

Nagle Algorithm

- ▶ It aims to improve the efficiency of TCP/IP networks by reducing the number of packets sent
- ▶ RFC 896 describes the "small packet problem"
 - ▶ An application repeatedly sends one-byte data
 - ▶ Each packet has a 40-byte header (20 bytes for TCP, 20 bytes for IPv4), meaning 41 bytes are needed to send 1 byte
 - ▶ This often occurs with Telnet, where each keystroke triggers the transmission of a one-byte character

Nagle Algorithm

- ▶ Combine a number of small messages and sends them all at once
- ▶ If the acknowledgment (ACK) of a previously sent packet has not been received, the sender buffers other packets until the packet has a suitable size for transmission

Nagle Algorithm

if data is ready for sending

if window size $\geq$ MSS and data to be sent $\geq$ MSS

send segment MSS

else

if there is a data for which no ack has been received yet

append data in the buffer until ack is received

else

send data immediatly

- ▶ MSS=Maximum Segment Size
- ▶ Nagle algorithm integrates poorly with the TCP delayed acknowledgments functionality (ACK responses may be combined together into a single response, reducing protocol overhead)